

Bachelor AI - MLOps

13 STREAM PROCESSING

Tobias Mérinat

`tobias.merinat@hslu.ch`

HSLU Hochschule
Luzern

DEPARTMENT OF INFORMATION TECHNOLOGY
Lucerne University of Applied Sciences and Arts
6343 Rotkreuz, Switzerland

September 17/24 and October 15 2026

- Also called *Event Stream Processing*
- Input and output objects of calculations are *sequences of event notifications over time*
- Calculation begins as soon as an event occurs

Comparison with Batch Processing

- Batch processing collects data, calculation occurs periodically
- Batch processing is efficient (parallelism can be exploited)
- Stream processing is fast (processing immediately after occurrence)
- Stream processing is effective (only necessary calculations are executed)

- Streaming Transport (Kafka, Redpanda, AWS Kinesis, GCP Dataflow, . . .)
- Stream Computing Engine (Apache Flink, Spark Streaming, Quix, . . .)

- Stream Processing is complex
- It differs fundamentally from Batch Processing
- Working with streams requires a mental shift
- In the following, some challenges

- Data is *unbounded*, there is no defined start and no end
- If you want to aggregate, you must define *Windows*
 - *Tumbling, Hopping, Sliding, Sessions, Snapshot Windows*
- Events can arrive late. How long do we wait?
- Order is not guaranteed
- Duplicates can occur

- Due to variable arrival rates, memory spikes can occur
- Aggregations can run for a long time, the entire window must be held in memory
- Memory consumption is therefore dependent on
 - the size of the window,
 - the event frequency
 - the body size

- The choice of window type and size
- Features are by definition only accurate once the entire window has been filled
- Tradeoff between actuality, accuracy, memory and CPU
- Joins across streams and time are complex

- The **CAP Theorem** states:
 - In distributed systems there is a tradeoff between availability and consistency
 - Only two of *Consistency*, *Availability* and *Partition Tolerance* can be guaranteed continuously

What Slows the Adoption of Stream Processing

According to a series of **interviews** conducted by Chip Huyen, the reasons why companies hesitate to build stream processing capabilities are the following:

- Companies see no advantage because
 - their infrastructure is not large enough for inter-service communication to become a bottleneck
 - they have no applications that would benefit from online prediction
 - although they have such applications, they are not aware of this due to lack of experience in this area
- High initial costs for new infrastructure
- High ongoing costs (in contrast to batch processing)
- In addition to Python, Java/Scala know-how is also necessary (Kafka, Flink, others)

However, streaming technologies have made rapid progress in recent years, which partially invalidates the last three points.

- Each event notification, i.e. each data point, flows through a series of transformations
- Such a data flow consists of
 - Source: Application, Sensor, Change Data Capture log, Social Media Feed, IoT Device, Queue
 - Transformations: enrich, filter, aggregate, modify
 - Sink: Database, Open Table Format, Queue, API, . . .

- For every operation that uses multiple rows (events), the cluster must be able to store a *State*, e.g. with:
 - Aggregations over windows
 - group by
 - joins

Each event notification has two types of timestamps:

- **Event Time:** When the event happened (recorded by the source system)
- **Processing Time:** When the event notification was processed (recorded by the processing system)

Watermarking:

- If the time between Event Time and Processing Time becomes too large, the event is late
- Using a *Watermark* we define how long after the end of a window we still wait for late events

- The source produces faster than event notifications can be processed
- Memory fills up slowly
- This is called *Backpressure*

- Standard joins join on an ID, without constraint on a timestamp
- Time-based joins additionally use the time aspect to join
- Join types: *Interval Joins*, *Temporal Joins*, *Lookup Joins*, *Window Joins*
- In each case, it is about limiting the state (memory consumption) and yet providing the output in a timely manner

- **Producer** and **Consumer**: The former send, the latter read messages
- **Topics**: A subject of interest, consumers subscribe to topics (*the highway from Bern to Zurich*)
- **Partition**: Subdivision of a topic (*lane with solid line*)
- **Key** of a message: Pins it to a partition, so messages cannot overtake each other
- **Consumer Group**: Members of a consumer group read the same topic, but from separate partitions. This means they do not receive the same events. Different consumer groups can read the same topic (messages are read multiple times).